

In hash maps, collisions are unavoidable.

Two different keys can produce the same hash/index.

JavaScript `Map` hides all this internally, but if you're learning DSA/interview-level hashing, you need to understand the collision strategies manually.

The 4 major collision handling techniques are:

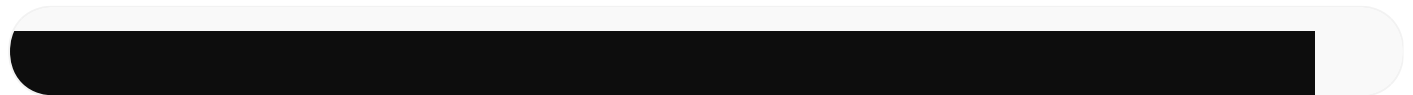
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Double Hashing

Let's dissect them properly. 🧠

1. Separate Chaining

The most common and easiest method.

Instead of storing one value at an index:



you store a bucket/list:



So collisions pile into the same bucket.

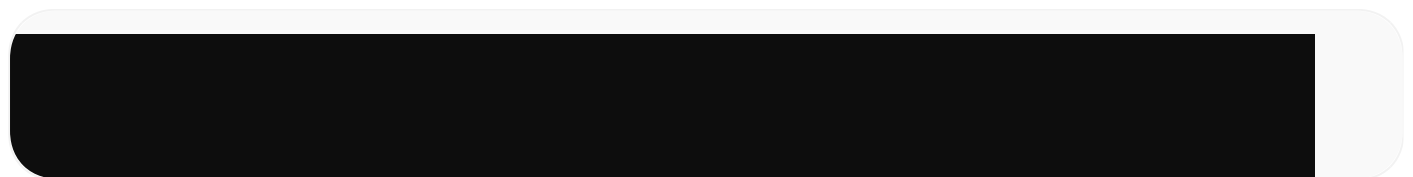
Example

Suppose table size = 5

Hash function:

```
</> JavaScript
index = key % 5
```

Insert:



All collide.

Visual

Implementation

</> JavaScript

```
class HashMap
    size = 5
    this. = new Array size . null .
    this. = size

    key
    return key % this.

    key value
    let index = this. key

    // update if exists
    for let pair of this. index
        if pair 0 === key
            pair 1 = value
            return

    this. index . key value

    key
    let index = this. key

    for let pair of this. index
        if pair 0 === key
            return pair 1

    return undefined

    console. this.
```

```
let hm = new HashMap

hm. 5 "apple"
hm. 10 "banana"
hm. 15 "mango"

hm.

console. hm. 10
```

Complexity

Case	Time
Best	$O(1)$
Worst	$O(n)$

Worst case happens when everything goes into one bucket.
Then your hash map degenerates into a linked list.

Real-world note

Java's `HashMap` uses chaining internally.

Modern implementations may convert long chains into balanced trees.

2. Linear Probing (Open Addressing)

Instead of storing multiple items in one bucket:

we search for the next empty slot.

Formula

where $i = 0, 1, 2, 3, \dots$

Example

Table size = 5

Insert:

Try next:

Store there.

Visual

Implementation

</> JavaScript

```
class LinearProbing
    size = 5
    this. = new Array size
    this. = size

    key
    return key % this.

    key value
    let index = this. key

    while this. index !== undefined
        index = index + 1 % this.

    this. index = key value

    key
    let index = this. key

    while this. index !== undefined
        if this. index 0 === key
            return this. index 1

    index = index + 1 % this.
```

```
return undefined
```

```
console.log(this)
```

```
let lp = new LinearProbing
```

```
lp.put(5, "apple")
```

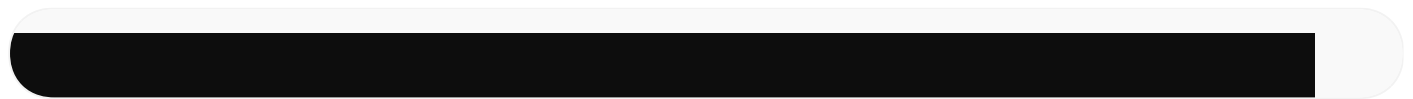
```
lp.put(10, "banana")
```

```
lp.put(15, "mango")
```

```
lp.get(5)
```

Major Problem → Primary Clustering 🦴

Clusters form:



Now new collisions keep piling nearby.

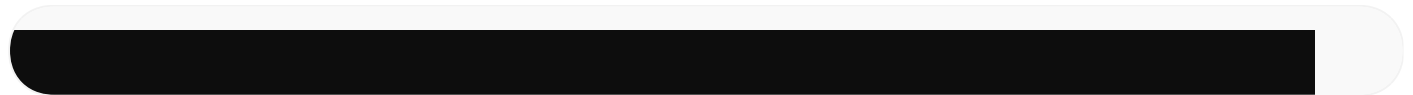
Performance degrades badly.

Complexity

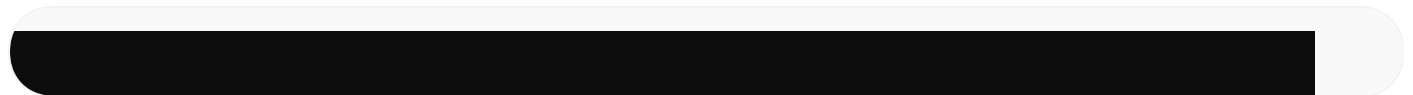
Case	Time
Average	$O(1)$
Worst	$O(n)$

3. Quadratic Probing

Linear probing walks like:



Quadratic probing jumps:



This reduces clustering.

Formula

Example

Collision at index 0:

Implementation

JavaScript

```
class QuadraticProbing
    size = 7
    this. = new Array size
    this. = size

    key
    return key % this.

    key value
    let index = this. key
    let i = 1

    while this. index !== undefined
        index = this. key + i * i % this.
        i++

    this. index = key value

    console. this.

let qp = new QuadraticProbing

qp. 10 "A"
qp. 17 "B"
qp. 24 "C"

qp.
```

Better than Linear?

Yes.

Because entries spread out more.

But:

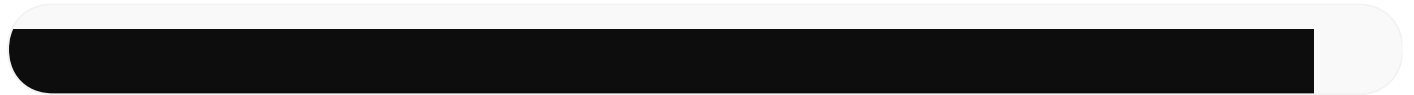
- still suffers clustering
- may fail to find empty slots even when table has space

That second issue confuses beginners badly.

Example Failure

Table size = 8

Quadratic sequence may only visit:



and never touch other cells.

That's why:

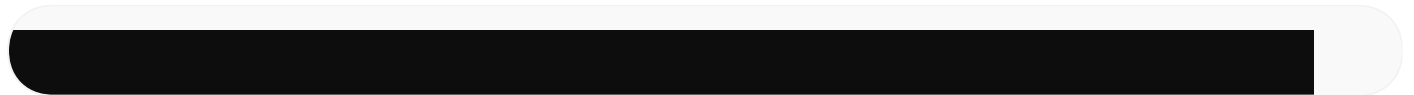
- table size often chosen as prime
- load factor kept low

4. Double Hashing

The most advanced among basic techniques.

Uses a SECOND hash function to determine jump size.

Formula



Why powerful?

Different keys get different jump patterns.

This almost kills clustering.

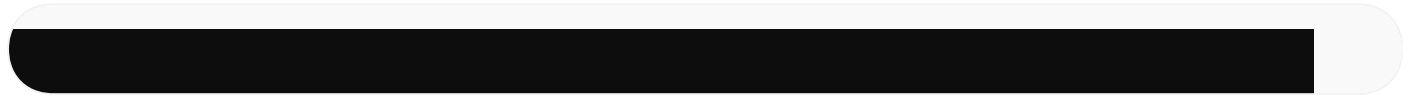
Example

`</>` JavaScript

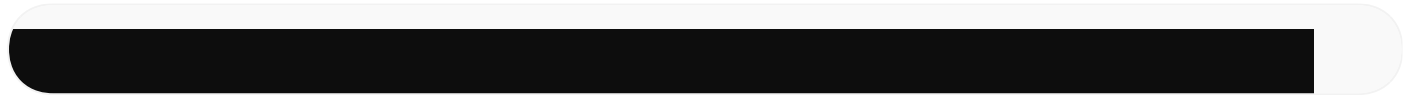
```
hash1 = key % size
hash2 = 7 - key % 7
```

Visual

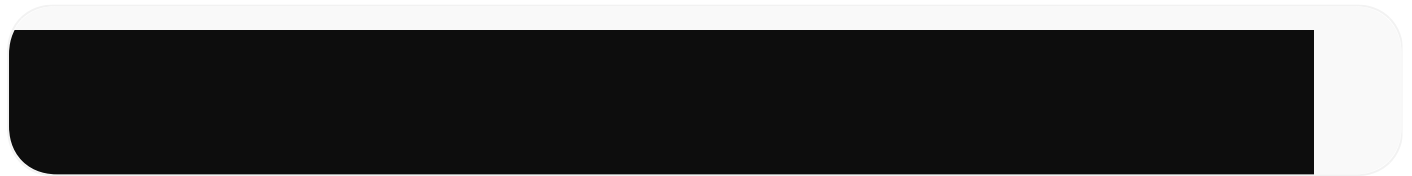
Instead of:



or



each key jumps uniquely:



Like every key driving its own weird taxi route through the parking lot 🚗🌀

Implementation

JavaScript

```
class DoubleHashing
  size = 7
  this. = new Array size
  this. = size

  key
  return key % this.

  key
  return 5 - key % 5

  key value
  let index = this. key
  let i = 0

  while this. index !== undefined
    index =
      this. key +
      i * this. key
      % this.
```

```
        i++

        this.        index = key value

        console.        this.

let dh = new DoubleHashing

dh.    10    "A"
dh.    17    "B"
dh.    24    "C"

dh.
```

Best Among Open Addressing?

Usually yes.

Because:

- less clustering
- better distribution
- closer to O(1)

But:

- more complex
- needs careful hash function design

Full Comparison

Method	Easy	Memory Efficient	Clustering	Performance
Chaining	✓	✗	None	Good
Linear	✓	✓	Very High	Medium
Quadratic	Medium	✓	Medium	Better
Double Hashing	Hard	✓	Low	Best

Important Realization

Most beginners memorize formulas.

Bad approach.

You need to understand:

Open Addressing

Means:

"If occupied, search another empty slot in SAME array."

Includes:

- Linear
- Quadratic
- Double hashing

Separate Chaining

Means:

"Store multiple values at same index."

Completely different philosophy.

What Real JavaScript Uses?

JavaScript objects and `Map` are engine-dependent.

V8 (Chrome/Node.js) uses highly optimized hash tables internally with:

- hidden classes
- dictionary modes
- optimized probing
- dynamic resizing

Not textbook implementations.

Your Next Step

Implement ALL 4 yourself from scratch without copy-paste.

Then add:

- `delete()`
- `resize/rehashing`
- `string hashing`
- `load factor`

That's where real understanding begins.